Übungsblatt 7: Moderne CPU-Architekturen

Mikrocomputertechnik 3 (MCT3)

Musterlösung

Zielsetzung

Sie vertiefen Vorlesung 7: Datenabhängigkeiten und Superskalarität, Register Renaming / ROB, Branch Prediction und Spekulation (inkl. Spectre) sowie Compiler-Optimierungen in Godbolt (`-O1` vs. `-O3`).

Bearbeitungszeit: ca. 45–60 Min. | **Orientierungspunkte:** Aufgabe 1: 8 P, Aufgabe 2: 12 P, Aufgabe 3: 4 P (Gesamt: 24 P)

Teil 1: Theorie — Von In-Order zu Out-of-Order

Aufgabe 1.1: Datenabhängigkeiten und Superskalarität

Superskalare Prozessoren nutzen Instruction-Level Parallelism (ILP) über mehrere Execution Units. Das größte Hindernis sind Datenabhängigkeiten.

Gegeben:

```
1: lw  t0, 0(s1)
2: add t1, t0, t2
3: sub t0, t4, t5
4: and t6, t0, t3
```

1. Identifizieren Sie eine *True Dependency* (Read-After-Write / RAW). Zwischen welchen Befehlen tritt sie auf, und warum kann die Hardware sie nicht parallelisieren?
2. Zwischen den Befehlen 2 und 3 liegt eine „falsche“ Abhängigkeit (Write-After-Read / WAR). Warum? Welches fatale Problem entstünde, wenn eine einfache CPU Befehl 3 vor Befehl 2 ausführen würde?

Musterlösung Aufgabe 1.1

1. **RAW** zwischen Befehl 1 (`lw t0, …`) und Befehl 2 (`add t1, t0, …`): Befehl 2 braucht den Wert in `t0`. Die Abhängigkeit ist real; ohne Ergebnis von 1 ist 2 nicht korrekt ausführbar.
2. **WAR** zwischen 2 und 3: Befehl 3 ist mathematisch unabhängig von 2, überschreibt aber dasselbe Architektur-Register `t0`. Würde Befehl 3 *schreiben*, bevor Befehl 2 den alten Wert aus `t0` *gelesen* hat, läse 2 den *neuen* Wert von `t0` statt den von `lw` — falsches Additions-Ergebnis. Die Schreib-/Lese-Reihenfolge auf dem Architektur-Register muss also gewahrt bleiben (oder per Renaming entkoppelt werden).

Aufgabe 1.2: Register Renaming und der ROB

Out-of-Order-(OoO-)CPUs lösen falsche Abhängigkeiten und führen Befehle asynchron aus.

1. Erklären Sie *Register Renaming*. Wie zerstört die Technik die WAR-Abhängigkeit zwischen Befehl 2 und 3 (Architektur- vs. physische Register)?
2. Warum sind Reorder Buffer (ROB) und *In-Order Commit* zwingend, um das System bei Interrupts vor inkonsistenten Zuständen zu schützen?

Musterlösung Aufgabe 1.2

1. Intern gibt es physische Register; die 32 RISC-V-Integernamen werden darauf abgebildet. `x0` bleibt fest 0. Befehl 2 liest die alte Version von `t0`. Befehl 3 erhält ein neues physisches Ziel. Die WAR-Kante entfällt dadurch. Ein gemeinsames Issue braucht zusätzlich freie Ports.

2. Jüngere Befehle können fertig sein, während ein älterer Load noch fehlt. Eine synchrone Exception an einem älteren Befehl darf dessen normalen Effekt nicht committen; jüngere Effekte werden verworfen. Ein Interrupt ist kein fehlerhafter Befehl, sondern wird an einer zulässigen Grenze angenommen. Im PRF-Modell stehen Werte in physischen Registern. Der ROB verfolgt die Ordnung. In-order-Retire macht den Architekturzustand sichtbar und ordnet den Speicher für andere Kerne nicht automatisch nach RVWMO.

Teil 2: Theorie — Branch Prediction und Spekulation

Aufgabe 2.1: Das Problem tiefer Pipelines

Moderne Prozessoren haben oft Pipelines mit weit über 15 Stufen.

1. Erklären Sie das *Control-Hazard*-Problem bei Sprüngen (`beq`). Warum führen Fehlprognosen bei tiefen Pipelines zu massiven Leistungseinbrüchen (*Misprediction Penalty*)?

Musterlösung Aufgabe 2.1

Bei `beq` weiß die Fetch-Stufe noch nicht, ob der Sprung genommen wird — die ALU entscheidet erst später. Die Pipeline muss aber jeden Takt füttern und spekuliert. Bei z. B. 15 Stufen können viele spekulative Befehle im Fließband liegen. Bei Fehlprognose: Flush → *Misprediction Penalty* (viele verlorene Takte, IPC-Einbruch).

Aufgabe 2.2: Branch History und Spekulation

1. Skizzieren Sie einen *2-Bit saturierenden Zähler*. Welchen Vorteil hat er gegenüber einem 1-Bit-Prädiktor bei verschachtelten Schleifen (Hysterese)?
2. Erklären Sie *spekulative Ausführung*. Was tut die CPU, während sie auf die ALU-Bestätigung des Sprungs wartet?

Musterlösung Aufgabe 2.2

1. Vier Zustände: `00` Strongly NT, `01` Weakly NT, `10` Weakly T, `11` Strongly T. Taken → +1 (cap `11`), Not Taken → −1 (floor `00`). **Hysterese:** Nach langer Taken-Schleife (`11`) und einem Abbruch fällt der Zähler nur auf `10` — beim nächsten Schleifenstart sofort wieder korrekt „Taken“. Ein 1-Bit-Prädiktor hätte nach dem Abbruch komplett umgeschaltet und falsch geraten.
2. Der Predictor wählt einen Pfad; die CPU wartet nicht passiv, sondern schiebt die vorhergesagten Instruktionen durch Renaming in die Execution Units und rechnet/lädt „auf Verdacht“. Bestätigt sich die Vorhersage, ist die Arbeit schon getan (höherer IPC).

Aufgabe 2.2b: 2-Bit-Prädiktor — Zustandstabelle

Ein 2-Bit saturierender Zähler startet im Zustand `11` (*Strongly Taken*). Kodierung: `00` Strongly NT, `01` Weakly NT, `10` Weakly T, `11` Strongly T. Vorhersage: Zustände `1x` → Taken, `0x` → Not Taken.

Gegeben die tatsächliche Sprungsequenz: **T, T, NT, NT, T**.

1. Füllen Sie eine Zustandstabelle: für jeden Schritt Startzustand, Vorhersage, tatsächliches Ergebnis, Treffer/Miss, Folgezustand.
2. Wie viele Treffer (korrekte Vorhersagen) ergeben sich insgesamt?

Musterlösung Aufgabe 2.2b

Schritt	Start	Vorhersage	Ist	Treffer?	Folge
1	`11`	T	T	ja	`11`
2	`11`	T	T	ja	`11`
3	`11`	T	NT	nein	`10`

Schritt	Start	Vorhersage	Ist	Treffer?	Folge
4	10	T	NT	nein	01
5	01	NT	T	nein	10

Treffer: 2 von 5 (Schritte 1 und 2).

Aufgabe 2.3: Seitenkanalangriffe (Spectre)

1. Bei Misprediction macht der ROB einen *Rollback* (Squash). Welcher Zustand wird dabei typisch *nicht* gelöscht und öffnet die Tür für Spectre?
2. Wie nutzt ein Angreifer Timing als Orakel, um eigentlich unerreichbare Daten zu rekonstruieren?

Musterlösung Aufgabe 2.3

1. Rollback verwirft spekulative Architektureffekte und stellt das gültige Mapping wieder her. Er löscht nicht jede Cache-Spur. Bei Spectre-v1 bleibt die Bounds-Prüfung architektonisch korrekt; der vorhergesagte Pfad läuft nur transient.
2. Das Geheimnis muss die Adresse eines folgenden Zugriffs beeinflussen. Nach dem Squash misst der Angreifer Zugriffszeiten. Ein schneller Hit ist ein verrauschtes Indiz, kein Beweis. KPTI trifft vor allem den ursprünglichen Meltdown-Mechanismus und ist keine universelle Spectre-Lösung.

Teil 3: Praxis — Performance-Analyse in Godbolt

Szenario: Der Compiler kann Code für die Pipeline vorbereiten. Nutzen Sie Compiler Explorer (godbolt.org).

Gegeben:

```
void vector_add(int *a, int *b, int *c, int len) {
    for (int i = 0; i < len; i++) {
        c[i] = a[i] + b[i];
    }
}
```

Aufgabe 3.1: Instruction Scheduling und Loop Unrolling

Öffnen Sie Godbolt, Compiler **RISC-V rv32gc gcc**, Code einfügen.

1. Kompilieren Sie mit `-O1`. Wo liegen typisch die `lw`- und `add`-Instruktionen in der Schleife?
2. Wechseln Sie auf `-O3`. Beobachten Sie:
    - *Instruction Scheduling:* Werden `lw`/`sw` so umsortiert, dass Load-Use-Stalls gemildert werden?
    - *Loop Unrolling:* Warum wird die Assembler-Ausgabe länger — und warum hilft das der Branch Prediction?

Dokumentieren Sie Ihre Beobachtungen in wenigen Sätzen.

Musterlösung Aufgabe 3.1

Godbolt zeigt die Übersetzung, keine OoO-Zyklen. `-O3` garantiert weder Unrolling noch eine kürzere Laufzeit. Beobachten Sie die tatsächlich erzeugten `lw`/`add`/`bne` und notieren Sie Compiler, Version und Flags.

Eine Reduktion bleibt eine RAW-Kette, bis der Compiler sie unter erlaubter Semantik aufteilt. Weniger Assemblerzeilen sind keine Laufzeit. Renode ohne Pipeline-Modell und Ripes mit einem skalaren Modell belegen kein Desktop-IPC.

Die Datei `labs/src/lab07_ilp.c` hält zwei funktional gleiche Reduktionen. Ein Host-GCC-Lauf prüft nur die Ergebnisgleichheit, nicht RISC-V-Scheduling.